Predictive Maintenance | Interim Project Report

Capstone Project 3 | Learner: ManjunathaNS
Scope: Data registration through experiment tracking
Dataset: AI4I 2020 Predictive Maintenance, UCI Machine Learning Repository

Executive overview

Unexpected equipment breakdowns disrupt production and create avoidable repair costs. This interim study builds a binary classifier for machine failure using operational sensor readings. The source dataset is synthetic, so measured performance is a research benchmark, not evidence of field deployment readiness.

Of six tuned algorithms, Bagging achieved the best five-fold training cross-validation F1 (0.776). On the held-out test set it reached F1 0.784, precision 0.860 and recall 0.721. It missed 19 of 68 failures; operators should treat this as a prioritisation aid, not an automatic shutdown decision.

1. Data Registration

Methodology

Registered the UCI AI4I 2020 CSV in the repository data folder and checked for all 14 expected columns using the validation script. Recorded shape, types, missing-value counts, duplicate count and target distribution before cleaning.

Measure	Result
Observations	10,000
Columns	14 (5 sensors, Type, 2 IDs, target, 5 failure flags)
Missing values / duplicate rows	0 / 0
Machine failures / non-failures	339 / 9,661
Failure prevalence	3.39%

Table 1. Dataset registration and quality summary

Source: UCI Machine Learning Repository, AI4I 2020 Predictive Maintenance Dataset (dataset 601): https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset. The data describes simulated industrial milling measurements; failure-mode flags record labels rather than deployable real-time sensor inputs.

2. Exploratory Data Analysis

Methodology: background and data overview

Inspected numeric summaries, target frequencies, sensor distributions, differences across failure labels and machine types, pairwise correlations and two-sensor relationships. Analyses below are descriptive associations, not causal claims.

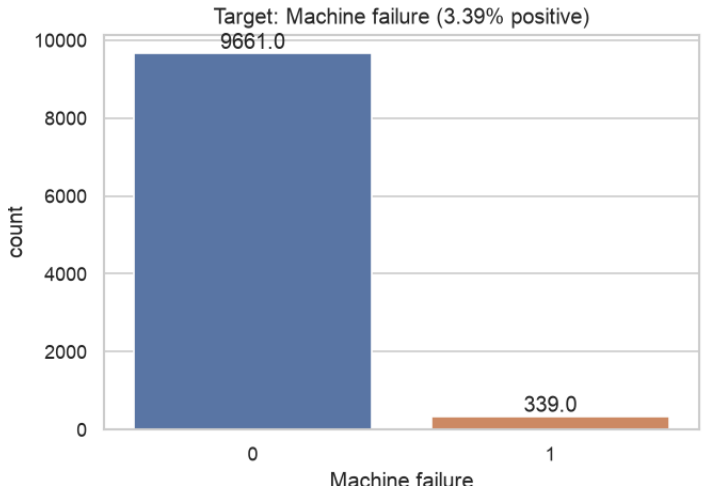


Figure 1. Failure versus non-failure record counts.

Only 3.39% of observations are failures. A model that always predicts no failure would achieve 96.61% accuracy while missing every breakdown. Consequently, positive-class recall and F1 are central business measures.

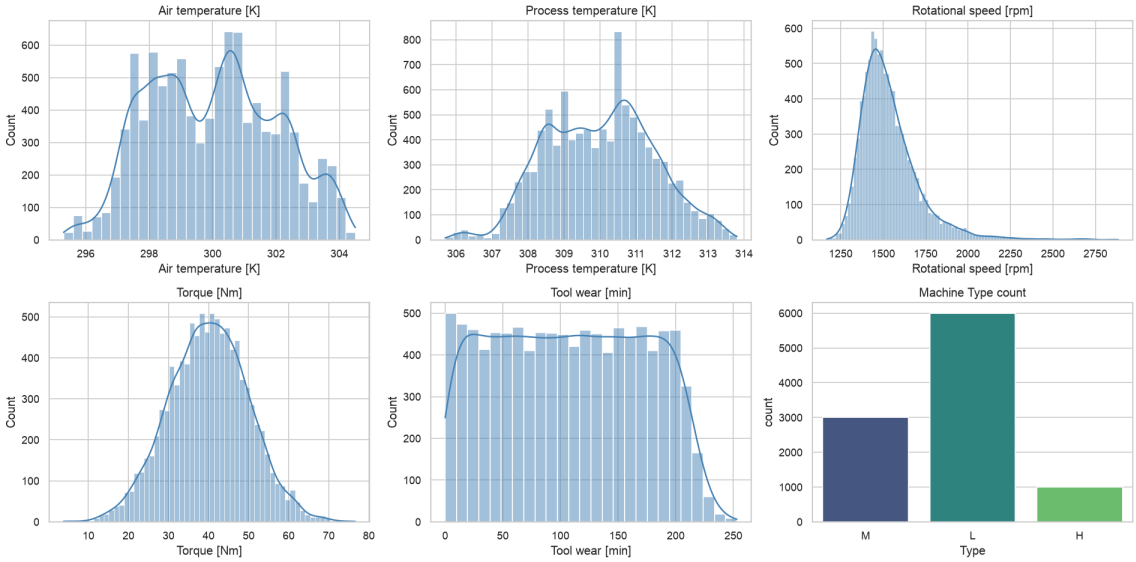


Figure 2. Univariate sensor measurements and machine type.

Machine type L has 6,000 of 10,000 records; M has 2,997 and H has 1,003. Sensor distributions span varied operating conditions; monitoring should check for drift in machine mix as well as in the numeric readings.

Methodology: bivariate and multivariate comparisons

Compared each numeric sensor across target groups; aggregated failure rates by machine type; inspected correlations and torque/tool-wear interactions. Failure-mode labels are presented only for context, never used as predictors.

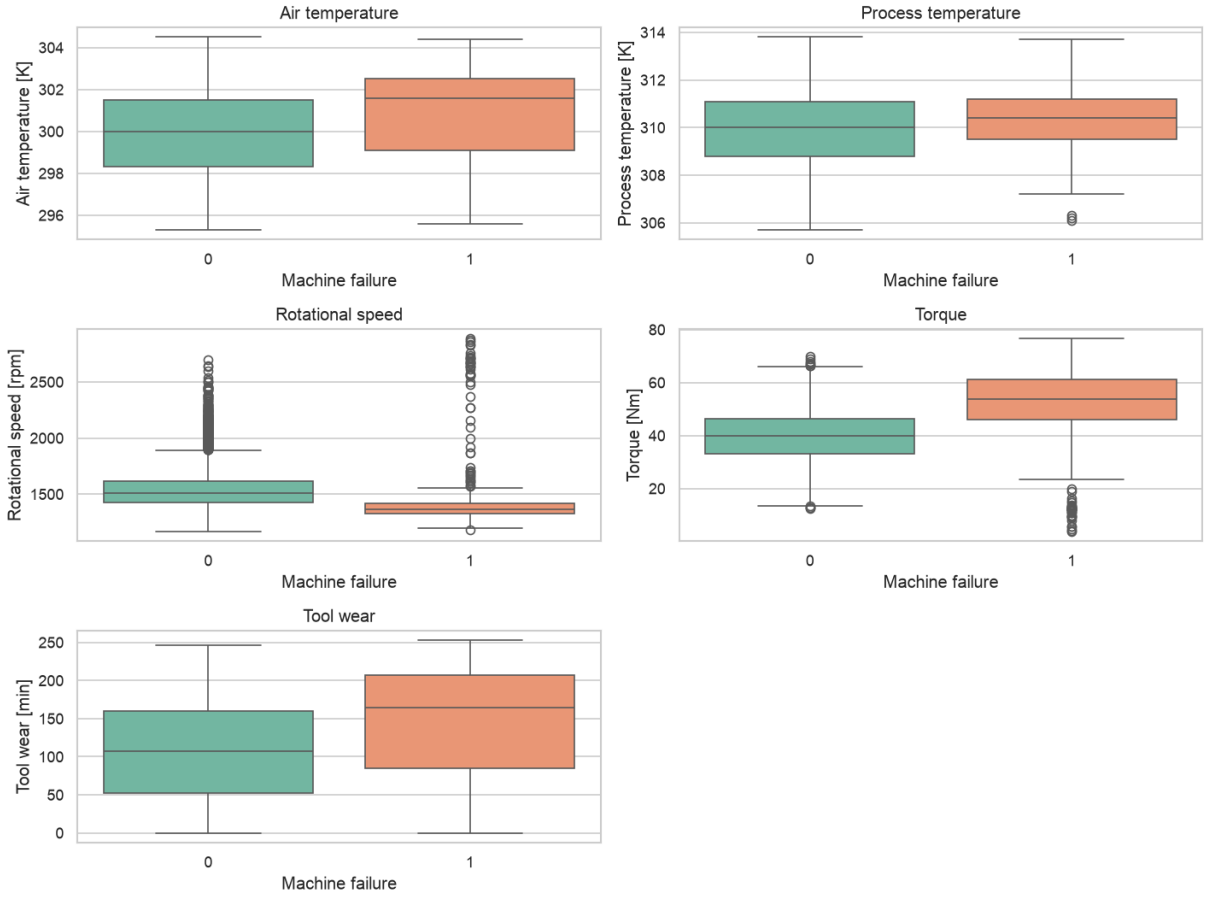


Figure 3. Sensor distributions conditional on failure.

Type	Records	Failures	Failure rate
H	1,003	21	2.09%
L	6,000	235	3.92%
M	2,997	83	2.77%

Table 2. Failure rate by machine type

Type L has the highest observed failure rate (3.92%), but this alone does not justify attributing failures to machine grade: operating conditions and the synthetic generation process can confound comparisons.

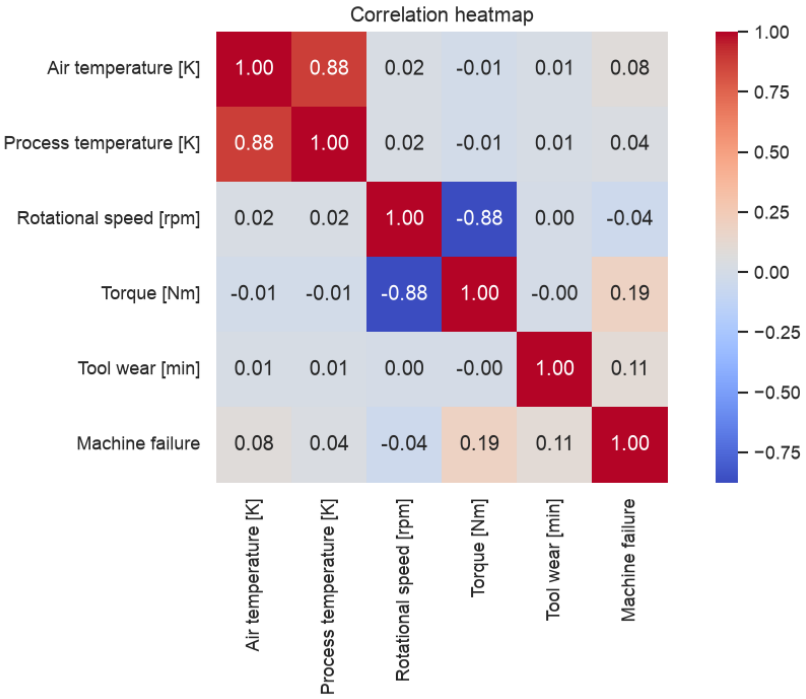


Figure 4. Correlations among sensors and the failure target.

Air and process temperatures covary strongly, so individual sensor effects must be interpreted cautiously. Pairwise correlation can also miss interactions relevant to overstrain or power-related failure.

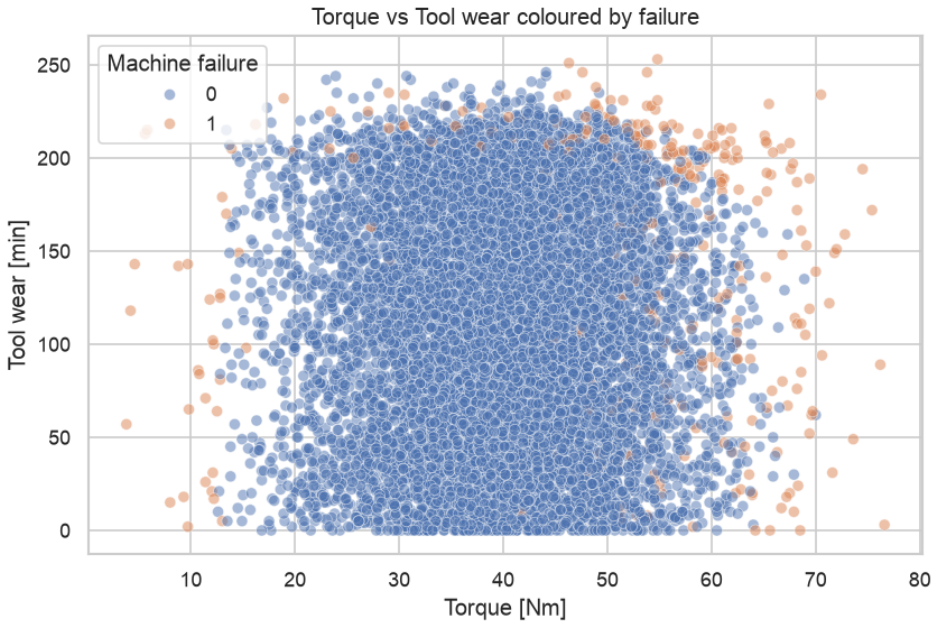


Figure 5. Torque and tool wear, coloured by failure.

The joint plot suggests operational context matters: no single torque or wear threshold is sufficiently reliable. Maintenance triage should use the full model score and prioritise validation on real production data.

3. Data Preparation

Methodology

Loaded the repository CSV directly. Removed 0 duplicate or missing rows, discarded UDI and Product ID identifiers, and removed TWF, HDF, PWF, OSF and RNF because they encode the outcome and would leak information into prediction. Normalised names and used a reproducible stratified 80/20 split (random seed 42). Standardisation and one-hot encoding occur within the model pipeline, fitted separately inside each training fold to avoid leakage.

Split	Rows	Failures	Stored as
Train	8,000	271	data/train.csv
Test	2,000	68	data/test.csv

Table 3. Prepared data and local workflow artifacts

In GitHub Actions, the first job validates/prepares data and uploads train.csv and test.csv as 'train-test-splits'; the second job downloads that artifact before tuning, then uploads and conditionally commits the selected model and metrics. This workflow is configured locally; a successful hosted run has not yet been verified.

4. Model Building and Experimentation Tracking

Methodology

Trained Decision Tree, Bagging, Random Forest, AdaBoost, Gradient Boosting and XGBoost with five-fold GridSearchCV optimising F1 on training data. The preprocessing transformer is part of each estimator pipeline. Selected the model by cross-validation F1, then reported held-out accuracy, precision, recall, F1 and ROC-AUC. Complete tuned parameters and confusion matrices are recorded in models/metrics.json.

Model	CV F1	Test precision	Test recall	Test F1	ROC-AUC
Decision Tree	0.701	0.836	0.676	0.748	0.874
Bagging	0.776	0.860	0.721	0.784	0.967
Random Forest	0.656	0.897	0.515	0.654	0.972
Adaboost	0.477	0.667	0.412	0.509	0.955
Gradient Boosting	0.743	0.873	0.706	0.780	0.968
Xgboost	0.746	0.887	0.691	0.777	0.977

Table 4. Five-fold tuning and held-out test results

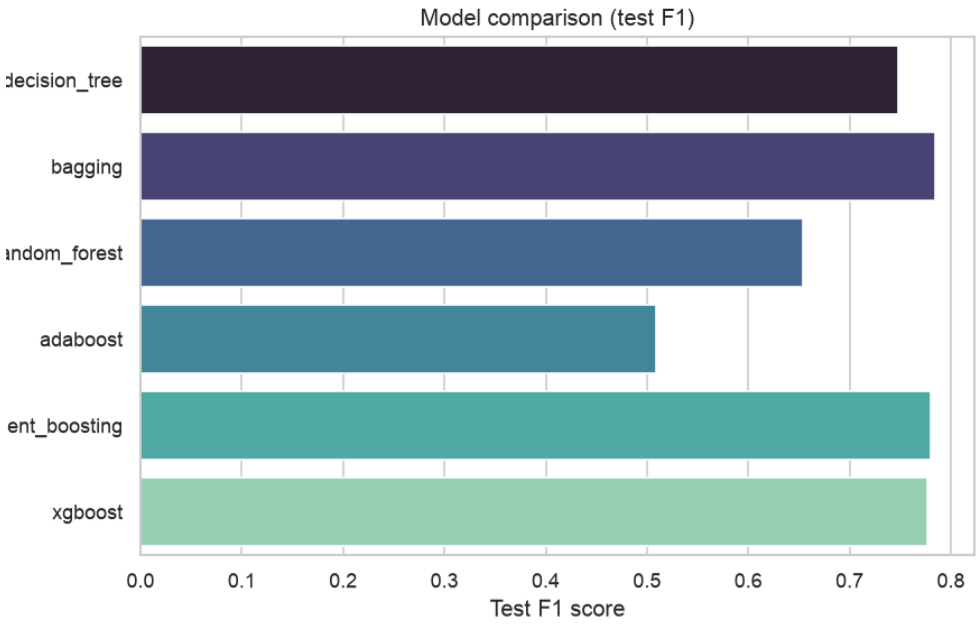


Figure 6. Held-out F1 comparison (not selection criterion).

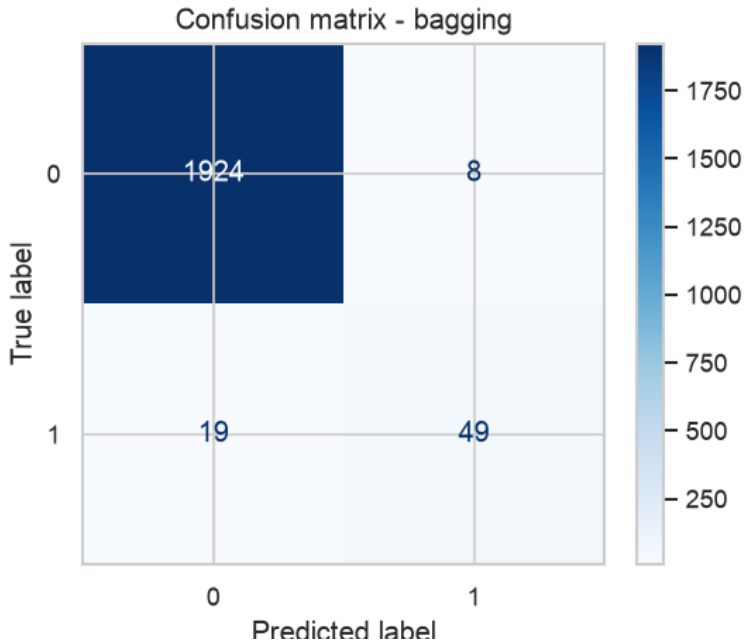


Figure 7. Bagging model: held-out confusion matrix.

Of 68 actual failures, the selected model detected 49 and missed 19, with 8 false alarms among 1,932 non-failures. A missed failure may cost more than an inspection triggered by a false alarm, so threshold choice should follow a measured cost-of-downtime analysis. Accuracy alone does not reflect this trade-off.

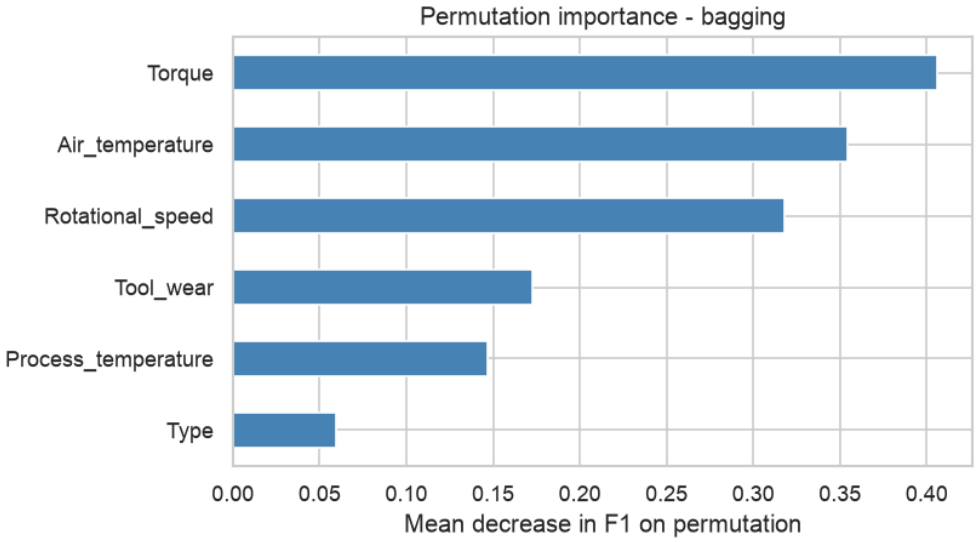


Figure 8. Permutation importance measured by change in F1 on held-out data.

Permutation importance shows association with predictive performance, not a causal effect; correlated sensors can divide their apparent contribution. Do not remove a low-ranked sensor without retraining.

5. Interim Business Implications and Limitations

The workflow establishes a reproducible benchmark and tracks all tuned settings, model metrics and a serialised model artifact. It does not demonstrate actual avoided downtime or operational cost savings. The dataset is synthetic and the split is random rather than chronological; validate on time-separated real equipment records before any deployment. Next phase: define an alert threshold and intervention costs, assess calibration and drift, run the hosted CI workflow and pilot with maintenance engineers.

Appendix A. Reproducibility and Supporting Outputs

The separately submitted executed HTML notebook contains the full Python code, charts, summary tables, tuned parameter printouts and model logs. Repository scripts: src/validate_dataset.py, src/prepare_data.py, src/train.py; CI job definitions: .github/workflows/pipeline.yml. Outputs: data/cleaned.csv, data/train.csv, data/test.csv, models/metrics.json and models/best_model.joblib. These artifacts provide the full evidence without raw code in the report body.

decision_tree: tuned parameters {'max_depth': 8, 'min_samples_split': 2}; CV F1=0.7007; test F1=0.7480; confusion matrix=[[1923, 9], [22, 46]].

bagging: tuned parameters {'max_samples': 1.0, 'n_estimators': 100}; CV F1=0.7760; test F1=0.7840; confusion matrix=[[1924, 8], [19, 49]].

random_forest: tuned parameters {'max_depth': None, 'n_estimators': 200}; CV F1=0.6565; test F1=0.6542; confusion matrix=[[1928, 4], [33, 35]].

adaboost: tuned parameters {'learning_rate': 1.0, 'n_estimators': 100}; CV F1=0.4772; test F1=0.5091; confusion matrix=[[1918, 14], [40, 28]].

gradient_boosting: tuned parameters {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}; CV F1=0.7433; test F1=0.7805; confusion matrix=[[1925, 7], [20, 48]].

xgboost: tuned parameters {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}; CV F1=0.7463; test F1=0.7769; confusion matrix=[[1926, 6], [21, 47]].

Source and citation: Matzka, S. (2020). AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C. All figures and tables in this report are generated from that dataset and this project's notebook.